



SRE LAB · CLASS EDITION

Running the Experiments

12 failure modes · 10 incident scenarios · one loop that works for all of them

Follow along with the cluster in front of you.

12

failure modes you can inject

10

scripted incident scenarios

6

steps in the loop, every time

Four parts, in order

Do not skip Part A. If the platform isn't verified, none of the exercises teach what they should.

A. Verify

Cluster, apps, and the four Datadog signals. Write down your baseline.

B. The loop

Six steps that work for every failure mode. The order is the skill.

C. Break it

12 failure modes in three families. One slide each.

D. Tell it

Map to the 10 scenarios, then explain it from both chairs.

The one rule: dashboard first, terminal second. If you opened a terminal before you opened a dashboard, you skipped the exercise.



PART A

Before you break anything

Five minutes that decide whether the next two hours teach anything.

Is the platform actually up?

```
kubectl config current-context      # must be the sre-lab EKS cluster
kubectl get pods -A                 # every app pod 1/1 Running
kubectl get hpa -A                  # cpu: N%/70% -- never <unknown>
kubectl top pods -n ecommerce       # proves metrics-server is serving

for a in ecommerce banking food-delivery student-portal support-tickets; do
  curl -s -o /dev/null -w "%{http_code} $a\n" "https://$a.$(cat .lab-domain)/"
done                                # five 200s
```

If kubectl looks wrong

Docker Desktop rewrites your context on start.
aws eks update-kubeconfig --name sre-lab --region us-east-1

If HPA says <unknown>

metrics-server isn't running. setup.sh step 9
installs it; cpu-spike can't work without it.

Do one real thing in each app

Traces only exist if somebody uses the app. Do these in a browser, not with curl.

APP	ACTION	LOGIN
ecommerce	Add to cart, then check out	--
banking	Log in, check a balance	demo123
food-delivery	Place an order	--
student-portal	View grades	demo123
support-tickets	File a ticket	--

Check all four signals in Datadog

This is the step people skip. It decides whether anything downstream is teachable.

SIGNAL	WHERE	GOOD LOOKS LIKE
Traces	APM > Traces, service:ecommerce-backend	browser -> backend -> Postgres
Logs	Logs, service:ecommerce-backend	Requests, with dd.trace_id injected
Events	Events, source:kubernetes kube_namespace:ecommerce	Kubernetes events present
Metrics	Dashboards > SRE Lab - ecommerce	All six widgets populated

If a trace starts at the backend: RUM didn't load. Check window.DD_RUM in the browser console -- the browser span is what makes it end-to-end.

Write down normal

Per app, from its dashboard, while everything is healthy.

SIGNAL	YOURS
Traffic (requests/sec)	_____
Error rate (%)	_____
p95 latency (ms) — not the average	_____
Memory vs the 256Mi limit	_____
Restart count	_____

Why this matters

Every scenario is graded on one question:

“Which widget moved first, and by how much?”

You can only answer it if you looked before anything broke.



PART B

The loop

Six steps. The same six for all twelve failure modes.

The experiment loop

Run this for every failure mode. The order is the whole skill.

1	Baseline	Look at the dashboard. Before, not after.
2	Inject	<code>./scripts/chaos/<script>.sh <app></code>
3	Datadog first	How big, and who? Then logs and events for the exact error.
4	Confirm as a customer	<code>curl</code> the real URL. Does it match what users report?
5	Only now, kubectl	READY column, describe pod, rollout history.
6	Undo, and prove it	<code>reset.sh</code> or <code>--undo</code> , then re-check the widget that moved.

Two things that will catch you out

Every app runs 2 replicas

A chaos script's single curl goes through the ALB to ONE random pod.

So effects are partial by design. To be deterministic, port-forward to a specific pod and POST to /api/chaos/* directly.

reset.sh can't fix a db-drop

That pod already failed readiness and left the Service, so the reset lands on a healthy one.

Reset it directly with kubectl exec — the command is on the db-drop slide.

Neither is a bug. Both are the lab behaving exactly like a real system, and both are worth ten minutes of discussion.



PART C

The 12 failure modes

Three families. The family tells you how to diagnose it, and how to undo it.

Three families

Twelve ways to break it, thirteen scripts — reset.sh is the undo tool, not a fault.

A • In-app conditions (5)

Toggled over HTTP against a live pod. No redeploy.

latency · errors · memory-spike · cpu-spike · db-drop

Undo: reset.sh

B • Kubernetes-level (2)

Bypass the app entirely.

kill-random-pod · scale-to-zero

Undo: kubectl

C • Change-driven (5)

Somebody shipped something. Deliberately quiet.

bad-deploy · break-config · rotate-secret · break-ingress · shrink-limits

Undo: --undo

First triage question, always: did something change, or did something drift? kubectl rollout history answers it in ten seconds.

A1 · Slow, but not failing

```
./scripts/chaos/inject-latency.sh ecommerce 4000
```

What you should see

p95 climbs hard
Error rate stays completely flat
The average barely moves

Where to look

The p95 widget — compare it with the average on purpose
curl https://ecommerce.<domain>/api/chaos shows latencyMs: 4000

Undo

```
./scripts/chaos/reset.sh ecommerce
```

The point

Slow is not failing. An error-rate monitor never fires here, which is why latency needs its own alert.

A2 · Failing fast

```
./scripts/chaos/inject-errors.sh banking 0.3
```

What you should see

Error rate jumps
Traces flagged as errors
5xx filling the logs

Where to look

Dashboard: Error Rate widget
Logs: service:banking-backend status:error

Undo

```
./scripts/chaos/reset.sh banking
```

The point

Errors and latency are two signals with two different responses. Don't collapse them into "it's broken".

A3 · The leak that heals itself

```
./scripts/chaos/memory-spike.sh support-tickets 300
```

What you should see

Memory climbs toward the 256Mi limit
A real OOMKilled, then a restart
Then it looks fine again

Where to look

describe pod | grep -A5 "Last State" -> Reason: OOMKilled
Dashboard: Pod Restarts, and the memory widget just before the kill

Undo

Nothing — the restart cleared the in-memory state.

The point

Self-resolving incidents get waved off. The saturation monitor exists to fire BEFORE the kill, not after.

A4 · Saturation, and the HPA

```
./scripts/chaos/cpu-spike.sh ecommerce 60
```

What you should see

Latency spikes on every endpoint that pod serves
The HPA scales out past 2 replicas

Where to look

kubectl -n ecommerce get hpa -w (run this first, in a second terminal)
Dashboard: p95 across all endpoints, not just one

Undo

Ends by itself; the HPA scales back after cooldown.

The point

Saturation shows up as latency everywhere, not as errors on one endpoint.

A5 · Running, but not Ready

```
./scripts/chaos/drop-db-connection.sh student-portal
```

What you should see

/readyz returns 503; /healthz stays 200

Pod goes NotReady after ~30s and leaves the Service

Nothing restarts — the process never crashed

Where to look

Logs: "readiness check failed: db connection dropped (chaos)"

kubectl get endpoints — the pod is gone from the list

Undo

```
kubectl exec -n <ns> <pod> — node -e  
"require('http').request({host:'localhost',port:4000,path:'/api/chaos/reset',method:  
'POST'}).end()"
```

The point

The most-asked interview question. Liveness and readiness gate different things — that's why there are two endpoints.

B1 and B2 · Bypassing the app

```
./scripts/chaos/kill-random-pod.sh banking  
./scripts/chaos/scale-to-zero.sh support-tickets support-tickets-frontend
```

B1 · kill-random-pod

Restart count +1, a brief blip if you're watching.

Self-heals — the Deployment reschedules immediately.

Point: alert on restart COUNT over a window, not one restart.

B2 · scale-to-zero

Full outage. The ALB returns 503 — no targets at all.

Undo: `kubectl scale deployment/... --replicas=2`

Point: compare this 503 with C4's 404. Empty and unhealthy target groups behave differently.

C1 · The tag that never existed

```
./scripts/chaos/bad-deploy.sh student-portal student-portal-backend student-portal-backend
```

What you should see

New pods ImagePullBackOff

Old pods keep serving

So the symptom is INTERMITTENT, not an outage

Where to look

describe pod -> "Failed to pull image ... not found"

rollout status -> stuck waiting for replicas that never come

Undo

```
kubectl -n student-portal rollout undo deployment/student-portal-backend
```

The point

The partial outage is RollingUpdate doing its job — it won't kill healthy pods for replacements that aren't ready.

C2 · The config that shipped fine

```
./scripts/chaos/break-config.sh banking
```

What you should see

One new pod never becomes Ready, then CrashLoops
The rollout stalls
The app still returns 200 — customers are fine

Where to look

Events: "Readiness probe failed ... connection refused" on :4000
logs <newest pod> -> "listening on 4001" <-- the penny drops

Undo

```
./scripts/chaos/break-config.sh banking --undo
```

The point

rollout undo does NOT fix this: the pod template never changed, only the ConfigMap it points at.

C3 · Rotated in one place

```
./scripts/chaos/rotate-secret.sh ecommerce
```

What you should see

New pod Running but 0/1, with ZERO restarts
No monitor fires at all
Endpoints list 2 addresses, not 3

Where to look

Logs: "readiness check failed: password authentication failed for user ..."
port-forward, then curl /healthz (200) vs /readyz (503)

Undo

```
./scripts/chaos/rotate-secret.sh ecommerce --undo
```

The point

The working credential now exists only in the memory of two pods that can't be replaced. Zero impact, high urgency.

C4 · Green dashboards, angry customers

```
./scripts/chaos/break-ingress.sh food-delivery
```

What you should see

A 404 from the BACKEND — not a gateway error
X-Powered-By: Express in the headers
/api/* still returns 200
Every dashboard stays green; backend throughput RISES

Where to look

```
curl -i https://food-delivery.<domain>/ | head -12
```

aws elbv2 describe-target-health ... (target health is in AWS, not Datadog)

Undo

```
./scripts/chaos/break-ingress.sh food-delivery --undo
```

The point

The best one in the lab: a total outage your monitoring cannot see, because every instrument sits inside a service that isn't broken.

C5 · The ceiling that moved

```
./scripts/chaos/shrink-limits.sh support-tickets
```

What you should see

New pods OOMKilled during startup (exit 137)
CrashLoopBackOff; old pods keep serving
Identical symptom to A3 — opposite cause

Where to look

rollout history, and compare the resources block with another app
Dashboard: memory usage never grew. The LIMIT moved.

Undo

```
./scripts/chaos/shrink-limits.sh support-tickets --undo
```

The point

rollout undo DOES work here (the pod template changed), unlike C2 and C3.
Explaining that difference is worth more than the fix.



PART D

Putting it together

From a broken pod to a story a hiring manager wants to hear.

The 10 scenarios

Each wraps one or more failure modes in a briefing with no answer attached.

#	SCENARIO	APP	STARTS WITH	USES
01	The Silent Checkout	ecommerce	condition	A1 + A2
02	Payday Panic	banking	condition	A5
03	The Stuck Order	food-delivery	condition	Redis to 0
04	Grades Gone Missing	student-portal	change	C1
05	The Midnight Memory Leak	support-tickets	condition	A3
06	The Noisy Neighbor	ecommerce + banking	condition	manual pg_sleep
07	The 4 PM Config Change	banking	change	C2
08	The Rotated Password	ecommerce	change	C3
09	Green Dashboards, Angry Customers	food-delivery	change	C4
10	The Tightened Limit	support-tickets	change	C5

PART D

A running order that builds

Roughly 20-30 minutes each, with discussion.

	RUN	WHY HERE
1	A1 on support-tickets	Simplest app, cleanest signal. Teaches p95 vs average immediately
2	A5 on banking	Running vs Ready — the most-asked interview question
3	A3 on support-tickets	A self-healing incident; sets up the contrast for step 5
4	C2 on banking	First change-driven one: “what changed?” before “what's broken?”
5	C5 on support-tickets	Same OOMKill as step 3, opposite cause. The contrast IS the lesson
6	C4 on food-delivery	Finish here: total outage, every dashboard green

Between runs, and at the end

```
./scripts/chaos/reset.sh <app>           # clears Family A state
./scripts/chaos/<script>.sh <app> --undo  # reverts Family C

kubectl get pods -A | grep -v Running    # confirm clean before the next one
ls .chaos-backup/                        # empty = nothing still injected

./scripts/teardown.sh                    # when you're done -- this costs real money
```

Leaving it running costs \$150-250/month. Tearing down and rebuilding takes about twenty minutes, and is itself good practice.

PART D

Now tell it from both chairs

Same incident, two questions. Practise out loud, ninety seconds each.

The DevOps engineer

Owens the delivery path.

"What changed?"

Ends with: the change reverted, and a pipeline that can't ship that mistake again.

The SRE

Owens the service level.

"What did it cost, and who told us?"

Ends with: budget spent, a named detection gap, one prevention item with an owner.

If you can deliver both versions of one incident, you're ready for either interview. See docs/devops-vs-sre.md.